

Lecture# 14

Networking using Volley

By

Dr. Muhammad Bilal

Volley Library

- Volley is an HTTP library that makes networking for Android apps easier and most importantly, faster
- Volley offers the following benefits
 - Automatic scheduling of network requests
 - Multiple concurrent network connections
 - Transparent disk and memory response caching with standard HTTP cache coherence
 - Cancellation request API. You can cancel a single request, or you can set blocks or scopes of requests to cancel.
 - Strong ordering that makes it easy to correctly populate your UI with data fetched asynchronously from the network.

Volley Dependencies

- The easiest way to add Volley to your project is to add the following dependency to your app's build.gradle file:

```
dependencies {  
    ...  
    implementation 'com.android.volley:volley:1.1.1'  
}
```

Add Internet Permission

- To use Volley, you must add the **android.permission.INTERNET** permission to your app's manifest. Without this, your app won't be able to connect to the network.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.mynetworkingapp">

    <uses-permission android:name="android.permission.INTERNET" />

    ...
```

Send Simple Request using Volley

- At a high level, you use Volley by creating a **RequestQueue** and passing it **Request** objects
- The **RequestQueue** manages worker threads for running the network operations, reading from and writing to the cache, and parsing responses
- Requests do the parsing of raw responses and Volley takes care of dispatching the parsed response back to the main thread for delivery.

Volley: RequestQueue Class

- A request dispatch queue with a thread pool of dispatchers
- It's enqueue the given Request for dispatch using “add” callback, resolving from either cache or network on a worker thread, and then delivering a parsed response on the main thread.

RequestQueue queue

Volley: newRequestQueue

- Volley provides a convenience method `Volley.newRequestQueue` that sets up a `RequestQueue` for you, using default values, and starts the queue

```
RequestQueue queue = Volley.newRequestQueue(this);
```

Volley: StringRequest Class

- A canned request for retrieving the response body at a given URL as a String.
- StringRequest constructor

```
/**
 * Creates a new request with the given method.
 *
 * @param method the request {@link Method} to use
 * @param url URL to fetch the string at
 * @param listener Listener to receive the String response
 * @param errorListener Error listener, or null to ignore errors
 */
public StringRequest(
    int method,
    String url,
    Listener<String> listener,
    @Nullable ErrorListener errorListener) {
    super(method, url, errorListener);
}
```


Volley: Request Class

- Base class for all network requests
- Following are predefined request codes in Request class

```
/** Supported request methods. */  
public interface Method {  
    int DEPRECATED_GET_OR_POST = -1;  
    int GET = 0;  
    int POST = 1;  
    int PUT = 2;  
    int DELETE = 3;  
    int HEAD = 4;  
    int OPTIONS = 5;  
    int TRACE = 6;  
    int PATCH = 7;  
}
```

Volley: Response Class

- Encapsulates a parsed response for delivery.

```
/**
 * Encapsulates a parsed response for delivery.
 *
 * @param <T> Parsed type of this response
 */
public class Response<T> {

    /** Callback interface for delivering parsed responses. */
    public interface Listener<T> {
        /** Called when a response is received. */
        void onResponse(T response);
    }

    /** Callback interface for delivering error responses. */
    public interface ErrorListener {
        /**
         * Callback method that an error has been occurred with the provided error code
         * optional user-readable message.*/
        void onErrorResponse(VolleyError error);
    }
    ...
}
```

Volley: StringRequest Implementation

```
// Request a string response from the provided URL.
StringRequest stringRequest = new StringRequest(Request.Method.GET, url,
    new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            Log.d("Main", "Response is " + response);
        }
    },
    new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Log.d("Main", "No response");
        }
    }
);
```

Volley: Adding Request in RequestQueue

```
// Instantiate the RequestQueue.
RequestQueue queue = Volley.newRequestQueue(this);
String url = "https://www.google.com";

// Request a string response from the provided URL.
StringRequest stringRequest = new StringRequest(Request.Method.GET, url,
    new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            Log.d("Main", "Response is" + response);
        }
    },
    new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Log.d("Main", "No response");
        }
    }
);

// Add the request to the RequestQueue.
queue.add(stringRequest);
```